

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

EMPACOTAMENTO DE CÓDIGO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

No desenvolvimento de software, muitas vezes é interessante conseguir distribuir um mesmo código para várias aplicações de forma organizada. Isto permite que as aplicações reusem código, facilitando sua comunicação e manutenibilidade.

Nesta aula, vamos aprender como fazer isso criando um pacote Python.

EMEND

HANDS ON

Nesta aula, vamos criar um pacote Python servirá como interface para deploy de modelos de aprendizado de máquina. O código está disponível no github na Branch: https://github.com/rocaabrera/curso_mlops/tree/empacotamento_codigo.



SAIBA MAIS

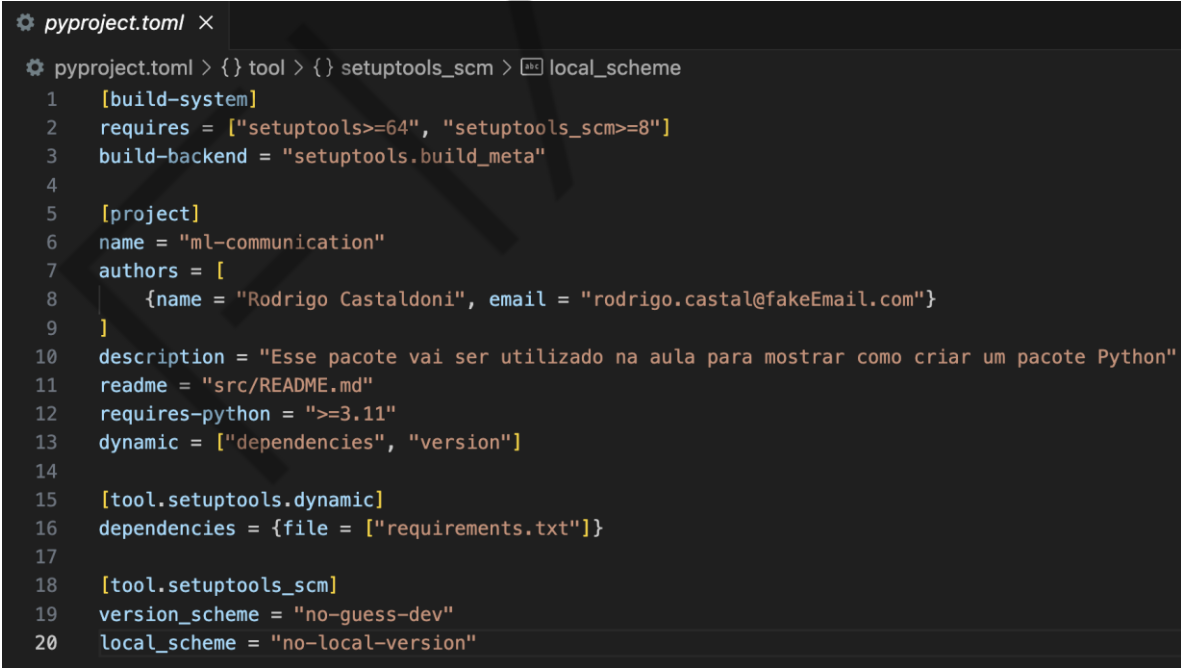
Inicialmente, visando evitar conflitos entre dependências de outros projetos Python, vamos criar um ambiente virtual. Uma das formas de fazer isso é utilizando o seguinte comando:

```
python -m venv <env_name>
```

Para entrar no ambiente virtual, você pode utilizar o comando (Mac ou Linux):

```
source <env_name>/bin/activate
```

Agora, vamos conversar sobre o principal arquivo desse tema, o **pyproject.toml**. A partir da PEP 621, a comunidade Python estabeleceu seu uso como a forma padrão de especificar metadados de um projeto. O arquivo `pyproject.toml` é separado por seções, denominadas tabelas. A Figura 1 mostra um exemplo de arquivo:



```
pyproject.toml ×
pyproject.toml > {} tool > {} setuptools_scm > [alt] local_scheme
1  [build-system]
2  requires = ["setuptools>=64", "setuptools_scm>=8"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "ml-communication"
7  authors = [
8    {name = "Rodrigo Castaldoni", email = "rodrigo.castal@fakeEmail.com"}
9  ]
10 description = "Esse pacote vai ser utilizado na aula para mostrar como criar um pacote Python"
11 readme = "src/README.md"
12 requires-python = ">=3.11"
13 dynamic = ["dependencies", "version"]
14
15 [tool.setuptools.dynamic]
16 dependencies = {file = ["requirements.txt"]}
17
18 [tool.setuptools_scm]
19 version_scheme = "no-guess-dev"
20 local_scheme = "no-local-version"
```

Figura 1 - Exemplo de arquivo `pyproject.toml`
Fonte: Elaborado pelo autor (2024)

A tabela `[build-system]` é utilizada para declarar qual backend (ferramenta) de construção será utilizada para construir o pacote (nesta aula vamos utilizar o

setuptools, que normalmente já vem instalado nos ambientes virtuais). Além disso, você também pode passar dependências específicas para construção do seu pacote. A tabela **[project]** é o formato usado pela maioria dos backends de construção, e é utilizada para especificar os metadados básicos do seu projeto, por exemplo: nome, autores, dependências, entre outros. Além disso, também existe uma tabela específica denominada **[tool]**, que possui sub-tabelas específicas para cada ferramenta, por exemplo **[tool.bandit]**, **[tool.ruff]**, **[tool.pytest.ini_options]**, **[tool.setuptools_scm]**, entre outros, conforme mostrado na Figura 1.

Com esse arquivo em mãos, vamos desenvolver o pacote. Coloque na raiz do repositório o folder **src**, e dentro, o folder com o seu pacote (no nosso caso **ml_communication**). Esse é um padrão utilizado na indústria, conforme ilustra a Figura 2:

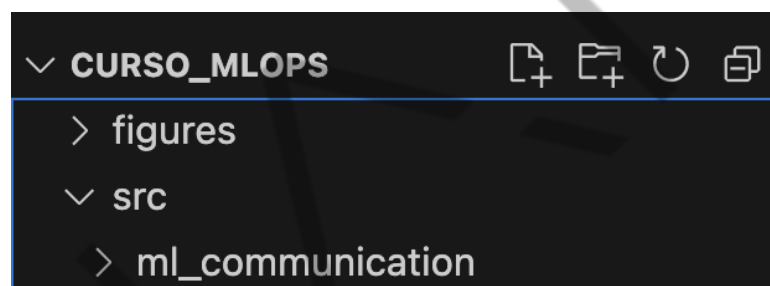


Figura 2 - Folder src e dentro dele o folder do pacote
Fonte: Elaborado pelo autor (2024)

IMPORTANTE: o nome do projeto deve ser único no índice onde ele será registrado (isso será explicado mais à frente) por exemplo, o PyPI.

Dessa forma, se você quiser reproduzir a aula, poderá encontrá-lo facilmente.

Coloque um sufixo ou prefixo com o seu nome para diminuir a chance de conflitos, exemplo: **ml_communication_rodrigo**.

Perceba que o nome do folder do seu pacote representa a forma como você vai importá-lo, no nosso caso:

```
import ml_communication
```

Durante a fase de desenvolvimento do seu pacote, instale no modo editável (você precisa estar no mesmo folder do arquivo pyproject.toml):

```
pip install -e .
```

Caso você não esteja no folder correto, você vai receber um aviso de erro parecido com este:

```
ERROR: file:///<path> does not appear to be a Python project: neither  
'setup.py' nor 'pyproject.toml' found.
```

Quando você realiza a instalação dessa forma, toda alteração feita é refletida de forma automática e você não precisa fazer o build do pacote para testar as modificações.

Depois de desenvolver o pacote, podemos gerar sua distribuição, processo conhecido como “fazer o build”. Para isso, precisamos instalar uma lib chamada build:

```
pip install -U build
```

Depois executar o comando:

```
python -m build
```

Será gerado um folder **dist** com os seguintes arquivos:

- Um arquivo .whl: representa a distribuição com build já realizado.
- Um arquivo .tar.gz: representa o código fonte, de forma que o usuário possa fazer o build manualmente após download.

Essa diferença acontece, pois existe a possibilidade de o pacote utilizar algo específico do sistema operacional onde a dependência vai ser executada. Por exemplo, a biblioteca do pytorch tem whl diferentes a depender do sistema operacional onde será utilizado, conforme mostra a Figura 3:

Built Distributions	
 torch-2.4.0-cp312-none-macosx_11_0_arm64.whl (62.1 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.12 macOS 11.0+ ARM64
 torch-2.4.0-cp312-cp312-win_amd64.whl (197.8 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.12 Windows x86-64
 torch-2.4.0-cp312-cp312-manylinux2014_aarch64.whl (89.7 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.12
 torch-2.4.0-cp312-cp312-manylinux1_x86_64.whl (797.2 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.12
 torch-2.4.0-cp311-none-macosx_11_0_arm64.whl (62.1 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.11 macOS 11.0+ ARM64
 torch-2.4.0-cp311-cp311-win_amd64.whl (197.9 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.11 Windows x86-64
 torch-2.4.0-cp311-cp311-manylinux2014_aarch64.whl (89.9 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.11
 torch-2.4.0-cp311-cp311-manylinux1_x86_64.whl (797.3 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.11
 torch-2.4.0-cp310-none-macosx_11_0_arm64.whl (62.1 MB view hashes)	Uploaded Jul 24, 2024 CPython 3.10 macOS 11.0+ ARM64

Figura 3 - Exemplo de distribuição do pacote
Fonte: Elaborado pelo autor (2024)

Nesta aula, vamos fazer o upload do nosso pacote no index de Test do PyPI, denominado PyPI (instância separada do índice de pacotes destinada a testes e experimentação). O index nada mais é que um banco de artefatos, o qual conterà os arquivos de distribuição. Vamos fazer os seguintes passos agora:

- Crie uma conta no TestPyPI neste [link](#).
- Verifique seu endereço de e-mail.
- Ative o MFA (Multi Factor Authentication) da sua conta.
- Crie um [token](#) de API do PyPI, definindo o escopo para a conta inteira.
- Adicione no path \$HOME/.pypirc o seguinte conteúdo:

```
[testpypi]
username = __token__
password = <TestPyPI API Token>
```

- Instale o pacote para fazer o **publish**:
 - `pip install -U twine`.
- Cheque se sua distribuição é válida (esse comando reportará problemas se não conseguir renderizar o README.md do pacote):

- `twine check dist/*`
- Faça o upload do pacote de distribuição no índice de teste:
 - `python -m twine upload --repository testpypi dist/*`

Após o Upload do pacote, você vai ter uma página no índice PyPI de Teste com o seu pacote, conforme mostra a Figura 4:

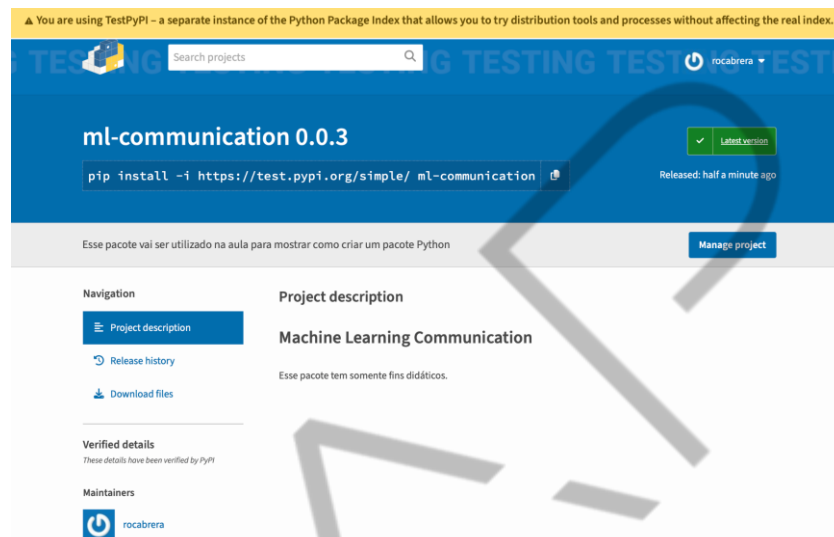


Figura 4 - Exibição da página após o upload do pacote
Fonte: Elaborado pelo autor (2024)

O nome da página será `https://test.pypi.org/project/ml-communication/`. Para saber mais sobre essa página, assista a videoaula.

Finalmente podemos baixar nosso pacote. Como estamos subindo um pacote no index de teste, não podemos utilizar o index default. Vamos indicar por meio do parâmetro `--index-url` qual url utilizar:

```
pip install --index-url https://test.pypi.org/simple/ <package-name>
```

No exemplo desta aula:

```
pip install --index-url https://test.pypi.org/simple/ ml-communication
```

As empresas normalmente utilizam um índice próprio, o qual pode ser construído com ferramentas como [JFrog](#).

Para finalizar esta aula, vamos conversar um pouquinho sobre versionamento. Todo pacote é versionado, e o padrão mais utilizado de versionamento é composto por três números separados entre si e por um ponto final, exemplo: `x.y.z`, onde `x`, `y` e

z são respectivamente a versão major, minor e patch. Esses números são alterados conforme a intenção que a pessoa desenvolvedora quer transmitir para quem utiliza seu pacote:

- major: você realiza uma mudança que não é retrocompatível com versões anteriores. Isto é, se alguém baixar o código novo, provavelmente ele não funcionará. Como exemplo, vamos mudar o nome de um parâmetro de uma função de interface.
- minor: quando uma funcionalidade nova é adicionada de forma retrocompatível. Como exemplo, implementar uma nova função.
- patch: quando é realizada a correção de alguma funcionalidade. Como exemplo, a correção de uma função que “quebrava” quando recebia uma string de parâmetro com um nome específico.

Perceba que essa padronização nos ajuda a entender quando houve uma mudança, exemplo:

- Se o pacote alterou da versão 0.0.1 para 0.0.2 você sabe que pode utilizar a versão 0.0.2 e seu código continuará funcionando da mesma forma.
- Ao atualizar o pacote da versão 0.0.1 para 0.1.0, você terá acesso a novas funcionalidades sem comprometer o funcionamento atual .
- Se o pacote alterou da versão 0.0.1 para 1.0.0, utilizar a versão 1.0.0 pode resultar em quebra de código.

Essa padronização explica porque vemos tantas vezes o arquivo requirements.txt instalando pacotes da seguinte maneira:

- pydantic==2.*

Entretanto, em aplicações (não desenvolvimento de dependências) pode fazer sentido fixar as dependências (pacotes) utilizadas.

Por último, a [PEP440](#) especifica como o versionamento deve ser feito e introduz o conceito de pacote com uma certa versão em desenvolvimento, também chamado de pre-release, como exemplo: 0.0.1.post0, 0.0.1.dev0 ou a combinação desses sufixos.

Nesta aula, vamos fazer o update de versão, processo denominado “bump”, utilizando um pacote chamado `setuptools_scm`. Basicamente, o pacote atualiza a versão patch toda vez que o build é feito e existe um arquivo com estado diferente de `unchanged` no repositório. Além disso, usamos tags do git para fixar as versões prontas para uso.

Caso seja do seu interesse, você pode fazer o processo manualmente mudando a versão do pacote no `pyproject.toml`, ou explorando um pacote alternativo (temos algumas possibilidades neste [link](#)).

As tags do git que vamos utilizar são as anotadas (annotated), as quais armazenam no banco de dados do Git um objeto inteiro representando o repositório naquele instante. Seguem os comandos que vamos utilizar:

Para adicionar uma tag annotated:

- `git tag -a <version> -m <message>`

Exemplos:

- `git tag -a v0.0.1 -m "version 0.0.1"`

Para deletar uma tag:

- `git tag --delete <version>`

O QUE VOCÊ VIU NESTA AULA?

Nesta aula você aprendeu a construir e distribuir um pacote Python para internet de acordo com a padronização feita pela comunidade. Além disso, também aprendeu como versionar um pacote.



REFERÊNCIAS

HOW to upload your package to the Python Package Index (PyPI) test server. **Florian Rathgeber** **web site**. 23 2020. Disponível em: <<https://kynan.github.io/blog/2020/05/23/how-to-upload-your-package-to-the-python-package-index-pypi-test-server>>. Acesso em: 10 set. 2024.

PACKAGING Python. **Writing your pyproject.toml**. 2024. Disponível em: <<https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>>. Acesso em: 10 set. 2024.

PEPS. **Storing project metadata in pyproject.toml**. 2024. Disponível em: <<https://peps.python.org/pep-0621/>>. Acesso em: 10 set. 2024.

PYTHON User Guide. **Packaging Python Projects**. 2024. Disponível em: <<https://packaging.python.org/en/latest/tutorials/packaging-projects/>>. Acesso em: 10 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: Pacotes. Reúso. Python.

EMUSE



POSTECH